

1. JSP (Java Server Pages)

1. JSP is a **server-side technology** that allows embedding **Java code into HTML pages**.
 2. It simplifies web development by separating **presentation logic (HTML)** from **business logic (Java)**.
 3. Each JSP is internally converted into a **Servlet** by the web container (like Apache Tomcat).
 4. JSP uses **directives, scriptlets, expressions, and declarations** for dynamic content generation.
 5. Common implicit objects:
 - request, response, session, application, out, pageContext, config, page, exception.
 6. JSP is ideal for **view layer** tasks — displaying data, handling forms, etc.
-

2. Servlet

1. A **Servlet** is a **Java program** that runs on the server and handles **client requests and responses**.
 2. It resides in the **javax.servlet** and **javax.servlet.http** packages.
 3. The servlet lifecycle includes:
 - `init()` – Initializes the servlet.
 - `service()` – Processes each request.
 - `destroy()` – Releases resources when the servlet is removed.
 4. It uses **HTTP methods** such as `doGet()` and `doPost()` to process web form data.
 5. Servlets are suitable for **controller logic** — performing computations, accessing databases, and sending data to JSPs.
-

3. URL Rewriting

1. **URL Rewriting** is a technique of appending **session information or parameters** to the URL. Example:

`http://example.com/dashboard?user=Kshitij`

2. Used when **cookies are disabled** in the browser.
 3. The server reads the parameters from the request and maintains user state across multiple pages.
 4. It is a client-visible method — data appears in the browser's address bar.
 5. **Advantages:** Simple, works even if cookies are off. **Disadvantages:** Less secure (data visible in URL).
-

4. Hidden Form Field

1. A **hidden form field** stores information inside an HTML form but is **not visible** to the user.

```
<input type="hidden" name="userID" value="101">
```
 2. The value is automatically sent to the server when the form is submitted.
 3. It is used to **preserve state** or **pass information between pages** without showing it in the URL.
 4. Suitable for small data like session IDs, tokens, or user choices.
 5. **Limitation:** Can be viewed or modified through page-source inspection, so not secure for sensitive data.
-

5. Cookies and Session Management

Cookies

1. A **cookie** is a small piece of data stored on the **client's browser** by the web server.
2. Used to maintain state between multiple HTTP requests (which are normally stateless).
3. Types:
 - **Non-persistent (Session Cookie):** Deleted when the browser closes.
 - **Persistent Cookie:** Stored until its expiry date.
4. In Servlets:

```
Cookie c = new Cookie("user", "Kshitij");  
response.addCookie(c);
```

5. On the next request, the browser sends it back via `request.getCookies()`.

Sessions

1. A **session** is a server-side mechanism to store user-specific information.

2. Each user gets a unique **Session ID** (via cookies or URL rewriting).

3. In Servlets:

```
HttpSession s = request.getSession();
s.setAttribute("user", "Kshitij");
```

4. Session tracking can be done through:

- Cookies
- URL Rewriting
- Hidden Form Fields
- HttpSession object

5. Sessions are **more secure and manageable** than client-side cookies for storing critical data.

Summary Table

Technique	Stored Where	Visibility	Common Use	Security
URL Rewriting	In URL	Visible	Passing IDs or data between pages	Low
Hidden Form Field	In HTML form	Hidden but viewable in source	Maintain form data	Low
Cookie	Client browser	Visible via browser tools	Remember login, preferences	Medium
Session (HttpSession)	Server	Hidden	User login state, transactions	High

Server.java

```
import java.io.*;
import java.net.*;

public class Server {
    public static void main(String[] args) throws IOException {
```

```
        ServerSocket ss = new ServerSocket(5002);
        System.out.println("Server waiting...");
        Socket s = ss.accept();
        System.out.println("Client connected.");

        BufferedReader in = new BufferedReader(new
InputStreamReader(s.getInputStream()));
        String msg;
        while ((msg = in.readLine()) != null && !msg.equalsIgnoreCase("exit"))
            System.out.println("Client: " + msg);

        s.close();
        ss.close();
    }
}
```

Client.java

```
import java.io.*;
import java.net.*;

public class Client {
    public static void main(String[] args) throws IOException {
        Socket s = new Socket("localhost", 5002);
        System.out.println("Connected to server.");
        BufferedReader in = new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out = new PrintWriter(s.getOutputStream(), true);

        String msg;
        while (!(msg = in.readLine()).equalsIgnoreCase("exit"))
            out.println(msg);

        s.close();
    }
}
```

How to run:

1. Compile both:

```
javac Server.java
javac Client.java
```

2. Run server first:

```
java Server
```

3. Then run client:

```
java Client
```

4. Type messages in the client — they'll appear on the server side.

5. Type `exit` to close the connection.